

CS791 Assignment2 [Team: MSD]

1. D3PM: equal contribution
2. DDPM: equal contribution

Deeksha Koul (24D0365)
Madhav Kotecha (24M0833)
Smita Gautam (24M0845)

September 23rd, 2025

1 D3PM

1.1 Introduction

Diffusion models are a class of generative models that generate images by gradually denoising a sample from a noise distribution. Discrete Denoising Diffusion Probabilistic Models (D3PMs) extend this idea to the discrete domain, modeling categorical distributions such as pixel intensities.

In this implementation, we applied D3PMs to the MNIST dataset, where each pixel is represented as an 8-bit discrete value in the range $[0, 255]$. The model is trained to reconstruct the original pixels from a progressively masked image using timestep conditioning.

1.2 Unconditional D3PM

1.2.1 Model Architecture

The D3PM model operates on discrete pixel values and learns the conditional distribution $p_\theta(x_0 | x_t, t)$, where x_0 is the original image and x_t is the image with some pixels masked at timestep t . The main components include:

- **Pixel Embedding:** Each pixel value is mapped to a continuous embedding vector of dimension 128.
- **Time Embedding:** Timesteps are encoded using sinusoidal embeddings projected to higher dimensions (512) to condition the model on the diffusion step.
- **Residual Blocks:** The network contains residual blocks arranged in a U-Net-like architecture for multi-scale feature extraction.

- **Classifier:** A 3×3 convolution maps the feature maps back to logits over 256 possible pixel values.

1.2.2 Residual Blocks

Each residual block consists of:

1. Two convolutional layers (3×3) with GroupNorm (8 groups) and ReLU activations.
2. Projection of the timestep embedding added to the first convolution output.
3. A shortcut connection; either identity if input and output channels match, or a 1×1 convolution if the channels differ.

1.2.3 U-Net Architecture (Shared)

The encoder-decoder U-Net structure is shared for both unconditional and conditional D3PM:

Encoder (Downsampling)

- **Down1:** ResidualBlock with 128 channels.
- **Down2:** ResidualBlock increasing channels to 256, spatial downsampling via MaxPool.
- **Down3:** ResidualBlock increasing channels to 512, spatial downsampling.

Bottleneck

- ResidualBlock with 512 channels, processing the deepest representation.

Decoder (Upsampling)

- **Up3:** ResidualBlock reducing channels to 256, concatenates skip connection from Down3.
- **Up2:** ResidualBlock reducing channels to 128, concatenates skip connection from Down2.
- **Up1:** ResidualBlock with 128 channels, concatenates skip connection from Down1.
- Final classifier outputs logits of shape $28 \times 28 \times 256$.

1.2.4 Time and Mask Conditioning

Time Embedding: Sinusoidal embeddings allow the network to encode timestep information.

Mask Scheduler: During training, pixels are progressively masked using a schedule:

- Linear schedule: mask probability increases linearly from 0.0 to 0.95.
- Cosine schedule: mask probability follows a cosine-based curve.
- Masked pixels are replaced with a special token (256).

1.2.5 Training and Sampling

Data Preparation

- MNIST dataset is loaded and normalized to $[0, 1]$.
- Images are converted to discrete integers $[0, 255]$ for compatibility with D3PM.

Forward Pass For each batch:

1. Sample a random timestep t .
2. Mask pixels according to the schedule.
3. Pass masked images and timestep embeddings through the D3PM to obtain logits.
4. Compute the cross-entropy loss between predicted logits and original pixels.

Optimization

- Optimizer: Adam with learning rate $1e - 4$.
- Batch size: 64.
- Number of epochs: 20.
- Model checkpoints are saved after each epoch, retaining the best model based on average loss.

Sampling Procedure Once trained, new images can be generated via iterative unmasking:

1. Initialize all pixels with the mask token (256).
2. For timesteps $t = T - 1 \rightarrow 0$:
 - (a) Predict logits for the current masked image.
 - (b) Convert logits to probabilities using softmax.
 - (c) Sample pixels from the categorical distribution.
 - (d) Update the image according to the mask scheduler, unmasking pixels gradually.
3. Normalize the final image to $[0, 1]$ for visualization.

1.3 Conditional D3PM

Conditional D3PM extends the discrete denoising diffusion model to incorporate class information, allowing generation of images conditioned on a specific label. In this implementation, MNIST digit classes are used as conditioning signals.

1.3.1 Model Architecture

The Conditional D3PM has the following additional components:

- **Class Embedding:** Each class label is mapped to a vector of the same dimension as pixel embeddings.
- **Combined Embedding:** Time and class embeddings are concatenated and projected through a linear layer with ReLU activation.

Other components (pixel embedding, residual blocks, U-Net, classifier) are shared with the unconditional model.

1.3.2 Forward Pass

For each batch:

1. Sample random timesteps t .
2. Mask pixels according to a mask scheduler (linear or cosine schedule).
3. Embed pixel values, timesteps, and class labels.
4. Concatenate and project embeddings to form the combined conditioning vector.
5. Pass masked images and embeddings through the shared U-Net to obtain predicted logits.
6. Compute cross-entropy loss with respect to the original unmasked images.

1.3.3 Training and Sampling

Data Preparation

- MNIST dataset is loaded and normalized to $[0, 1]$.
- Images are converted to discrete integers $[0, 255]$ for compatibility with D3PM.
- Class labels (0–9) are loaded as conditioning signals.

Forward Pass

For each batch:

1. Sample random timesteps t .
2. Mask pixels according to a mask scheduler (linear or cosine schedule).
3. Embed pixel values, timesteps, and class labels.
4. Concatenate and project embeddings to form the combined conditioning vector.
5. Pass masked images and embeddings through the shared U-Net to obtain predicted logits.
6. Compute cross-entropy loss with respect to the original unmasked images.

Optimization

- Optimizer: Adam with learning rate $1e - 4$.
- Batch size: 64.
- Number of epochs: 20.
- Model checkpoints are saved after each epoch, retaining the best model based on average loss.

Sampling Procedure

To generate images conditioned on a class label:

1. Initialize all pixels with the mask token (256).
2. For timesteps $t = T - 1 \rightarrow 0$:
 - (a) Predict logits conditioned on the class label.
 - (b) Convert logits to probabilities using softmax.
 - (c) Sample pixels from the categorical distribution.
 - (d) Update the image according to the mask scheduler, progressively unmasking pixels.
3. Normalize the final generated images to $[0, 1]$ for visualization.

2 DDPM

2.1 Unconditional Diffusion Model

2.1.1 Model Architecture

We have used a UNet with time embeddings normalised by T as our base diffusion model. It applies time-conditioned residual updates via linear projections added to feature maps at multiple scales, with GroupNorm and ReLU nonlinearities, and predicts noise in the same image channel space. The time step, t , is embedded through a small MLP into a vector that is projected and added at encoder and decoder stages. The architecture uses convolutional downsampling and transpose-convolution upsampling with skip connections, and the output is a 1-channel noise prediction via a 1×1 conv head.

2.1.2 Hyperparameter Optimization

The effects of T value were studied keeping all other hyperparameters fixed. We kept under consideration the following hyperparameters lbeta and ubeta, the lower and upper bounds of the linear noise schedule, denoted by l and u . Given T time steps, this initializes $0 = l$ and $T = u$, and

$$\beta_t = l + \frac{t}{T}(u - l), \quad t \in \{1, \dots, T - 1\}.$$

So, the α_t and $\bar{\alpha}_t$ can be written as

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{i=0}^t \alpha_i.$$

We should ensure that

$$\sqrt{\bar{\alpha}_t} \rightarrow 0 \text{ as } t \rightarrow T.$$

Batch Size: This denotes the size of the batch over which we do a gradient update while training. LR value: The Learning Rate for the Adam Optimiser used for updating the weights with the gradient. Schedule: The Noise schedule used for determining the t values. The hyperparameters fixed values are lbeta - 10^{-5} ubeta - 0.02 batch size - 128 lr value - $5 \cdot 10^{-5}$ schedule - Linear Schedule

2.1.3 Effect of Noise Schedule

Along with the linear noise schedule, we have also implemented the cosine noise schedules. Cosine Noise Schedule: We follow the implementation for the cosine noise schedule given in: We define

$$f(t) = \cos^2\left(\frac{\pi}{2} \frac{(t/T + s)}{1 + s}\right).$$

We use it to define $\bar{\alpha}_t$ as follows

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}.$$

Using which, we can obtain α_t and β_t by

$$\alpha_t = \frac{\hat{\alpha}_t}{\hat{\alpha}_{t-1}}, \quad t \geq 1, \quad \beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}.$$

For all the noise schedules, the values of α_t and β_t are clipped with a pre-defined clip min = 10^{-9} and clip max = 0.9999 for numerical stability.

2.2 Conditional DDPM

2.2.1 Classifier-Free Guidance

Conditional sampling and guided sampling are distinct techniques in diffusion models, each with different mechanisms for incorporating class information.

2.2.2 Conditional Sampling

Conditional sampling directly trains a model to generate samples conditioned on a specific label. The model is parameterised as $p(x | y)$, and is trained on paired data (x, y) to learn the class-specific distribution. Since the conditioning information is part of the models input, no external classifier or guidance mechanism is required. This method offers a straightforward way to generate class-specific samples but may struggle to balance sample diversity and fidelity in complex datasets.

2.2.3 Guided Sampling

To explicitly incorporate class information into the diffusion process, Dhariwal trained a classifier $f(y | x_t, t)$ on noisy image x_t and use gradients $\nabla_x \log f(y | x_t)$ to guide the diffusion sampling process toward the conditioning information e.g., a target class label by altering the noise prediction. Without an independent classifier, it is still possible to run conditional diffusion steps by incorporating the scores from a conditional and an unconditional diffusion model. Let the unconditional denoising diffusion model be $p(x)$, parameterized through a score estimator $\epsilon(x_t, t)$, and the conditional model parameterized through $\epsilon(x_t, t, y)$. These two models can be learned via a single neural network. Precisely, a conditional diffusion model $p(x | y)$ is trained on paired data (x, y) , where the conditioning information y gets discarded periodically at random so that the model knows how to generate images unconditionally as well, that is, $\epsilon(x_t, t) = \epsilon(x_t, t, y = \cdot)$.

$$p(x | c) = \frac{p(c | x)p(x)}{p(c)}$$

$$\Rightarrow \log p(x | c) = \log p(c | x) + \log p(x) - \log p(c)$$

$$\Rightarrow \nabla_x \log p(x | c) = \nabla_x \log p(c | x) + \nabla_x \log p(x)$$

conditional score = $\nabla_x \log p(c | x)$ classifier gradient + $\nabla_x \log p(x)$ unconditional score. For the Explicit conditioning term, we have

$$\nabla_x \log p(c | x_t) = \nabla_x \log p(x_t | c) - \nabla_x \log p(x_t).$$

$$\nabla_x \log p'(x_t | c) = \nabla_x \log p(x_t) + w \cdot \nabla_x \log p(c | x_t)$$

conditioning term

$$\Rightarrow \nabla_x \log p'(x_t | c) = \nabla_x \log p(x_t) + w \cdot (\nabla_x \log p(x_t | c) - \nabla_x \log p(x_t))$$

conditional score unconditional score. In terms of noise prediction using -thetas

$$\bar{\epsilon}(x_t, t, y) = \epsilon(x_t, t) + w(\epsilon(x_t, t, y) - \epsilon(x_t, t)).$$

2.2.4 Diffusion-based classifier

Conditional Generative model can be used for classification using Bayes Theorem on model predictions $p(x | c_i)$ and the prior $p(c)$ over labels $\{c_i\}$

$$p(c_i | x) = \frac{p(c_i) p(x | c_i)}{\sum_j p(c_j) p(x | c_j)}$$

(1) Introducing uniform prior over $\{c_i\}$ (i.e., $p(c_i) = \frac{1}{N}$) results in all of the $p(c)$ terms cancelling. Now, The Posterior distribution over $\{c_i\}_{i=1}^N$ can be approximated using

$$p(c_i | x) = \frac{\exp\{-E_{t,\epsilon}[\|\epsilon - \epsilon(x_t, c_i)\|^2]\}}{\sum_j \exp\{-E_{t,\epsilon}[\|\epsilon - \epsilon(x_t, c_j)\|^2]\}}$$

(2) This expectation can be approximated using Monte-Carlo estimate by sampling, N (t_i, ϵ_i) pairs, with $t_i \sim [1, n \text{ timesteps}]$ and $\epsilon \sim \mathcal{N}(0, I)$, and computing

$$\frac{1}{N} \sum_{i=1}^N \left\| \epsilon_i - \epsilon(\sqrt{\bar{\alpha}_{t_i}} x + \sqrt{1 - \bar{\alpha}_{t_i}} \epsilon_i, c_j) \right\|^2$$

(3) Finally, we select the class label with the maximum posterior probability

$$\arg \max_{c_i \in C} p(c_i | x)$$

which is equivalent to

$$\arg \min_{c_i \in C} \text{mean}(\text{Errors}[c_i])$$

Here,

$$\text{Errors}[c_i] = \{ \|\epsilon_j - \epsilon(\sqrt{\bar{\alpha}_{t_j}}x + \sqrt{1 - \bar{\alpha}_{t_j}}\epsilon_j, c_i)\|^2 \mid (t_j, \epsilon_j) \text{ samples} \}$$

The complete algorithm is presented in Algorithm 1. ... A comparison between a regular classifier ... and the implemented diffusion-based classifier is shown in Table 11. ... Overall, the diffusion-based classifier performs comparably to the MLP classifier across all three tested datasets Blobs, Circles, and Manycircles.

Direction and Declaration for Running generate.py

Add the desired model path using any of the following four paths: `ddpm_cond_path`, `ddpm_path`, `d3pm_path`, `d3pm_cond_path` (use one at a time).

Set the `num_steps` and `sampling_steps` parameters to be the same as those used during training.

Example command: `python generate.py --ddpm_cond_path model.pth --num_steps 500 --sampling_steps 500`

Generated samples will be saved under the folder: `"generated_samples"`.

After sample generation, run the following command to compute FID scores for evaluation:

```
python evaluation.py
```

Note: The model architecture was initially chosen based on the DDPM/D3PM papers and then carefully adapted to a large language model (LLM) framework by reviewing the architecture line by line.

3 Observations

scheduler	steps	FID
linear	500	110.2732
linear	1000	94.9468
linear	2000	79.9016
cosine	500	53.9523
cosine	1000	59.1636
cosine	2000	53.6793

Table 1: Unconditional D3PM

Observation: Cosine noise scheduling achieves better FID scores compared to linear scheduling at equivalent or fewer steps, indicating improved model performance.

scheduler	steps	FID
linear	500	72.6729
linear	1000	80.3356
linear	2000	106.9572
cosine	500	87.6433
cosine	1000	87.5597
cosine	2000	174.3908

Table 2: Unconditional DDPM

Observation: For DDPM, linear scheduling generally outperforms cosine scheduling in FID, especially at higher steps.

Observation: From the results for conditional DDPM, the linear scheduler consistently achieves much lower FID scores compared to the cosine scheduler across all step settings, indicating better image quality. For both schedulers, increasing the number of steps generally reduces the average FID.

Scheduler	Steps	FID (of each class)
linear	500	Class 0: 102.50 Class 1: 86.82 Class 2: 72.96 Class 3: 37.48 Class 4: 53.88 Class 5: 48.92 Class 6: 53.14 Class 7: 84.42 Class 8: 77.62 Class 9: 56.19 Avg: 67.39
linear	1000	Class 0: 124.06 Class 1: 110.86 Class 2: 92.70 Class 3: 59.02 Class 4: 91.24 Class 5: 62.31 Class 6: 88.25 Class 7: 107.07 Class 8: 87.90 Class 9: 70.46 Avg: 89.39
linear	2000	Class 0: 151.57 Class 1: 55.83 Class 2: 99.95 Class 3: 70.50 Class 4: 82.46 Class 5: 77.80 Class 6: 106.71 Class 7: 105.15 Class 8: 95.69 Class 9: 83.43 Avg: 92.91

Table 3: Conditional D3PM with linear scheduler

scheduler	steps	FID (of each class)
cosine	500	Class 0: 49.7333 Class 1: 49.4340 Class 2: 50.7023 Class 3: 35.0709 Class 4: 43.7206 Class 5: 40.0906 Class 6: 40.3698 Class 7: 45.3089 Class 8: 50.5925 Class 9: 43.9218 Avg: 44.8945
cosine	1000	Class 0: 54.8465 Class 1: 51.2903 Class 2: 58.1587 Class 3: 38.7375 Class 4: 47.8068 Class 5: 43.1509 Class 6: 47.2500 Class 7: 49.7593 Class 8: 48.5849 Class 9: 42.5577 Avg FID: 48.2143
cosine	2000	Class 0: 52.2921 Class 1: 50.3337 Class 2: 44.1936 Class 3: 33.2667 Class 4: 42.9053 Class 5: 42.3212 Class 6: 45.5347 Class 7: 52.0378 Class 8: 52.6987 Class 9: 44.6223 Avg FID: 46.0206

Table 4: Conditional D3PM with cosine scheduler

scheduler	steps	FID (of each class)
linear	500	Class 0 FID: 118.1036 Class 1 FID: 74.9864 Class 2 FID: 89.7843 Class 3 FID: 79.6934 Class 4 FID: 133.8833 Class 5 FID: 39.2396 Class 6 FID: 81.4469 Class 7 FID: 68.3135 Class 8 FID: 85.1069 Class 9 FID: 43.5922 Avg: 81.4150
linear	1000	Class 0 FID: 70.6783 Class 1 FID: 84.2323 Class 2 FID: 118.4549 Class 3 FID: 73.5312 Class 4 FID: 128.8344 Class 5 FID: 44.4670 Class 6 FID: 75.3088 Class 7 FID: 125.6261 Class 8 FID: 63.8434 Class 9 FID: 52.1536 Avg: 83.7130
linear	2000	Class 0: 63.2944 Class 1: 60.1915 Class 2: 105.4849 Class 3: 40.5800 Class 4: 68.4666 Class 5: 42.2785 Class 6: 73.6948 Class 7: 114.4938 Class 8: 80.5390 Class 9: 54.1815 Avg: 70.3205

Table 5: Conditional DDPM with linear scheduler

scheduler cosine	steps 500	FID (of each class) Class 0 FID: 434.7336 Class 1 FID: 544.6147 Class 2 FID: 365.3296 Class 3 FID: 307.9298 Class 4 FID: 362.4079 Class 5 FID: 337.4753 Class 6 FID: 394.8502 Class 7 FID: 399.0886 Class 8 FID: 366.6292 Class 9 FID: 257.8638 Avg FID: 377.0923
cosine	1000	Class 0 FID: 212.2514 Class 1 FID: 336.1784 Class 2 FID: 241.6692 Class 3 FID: 209.1329 Class 4 FID: 415.0370 Class 5 FID: 387.9117 Class 6 FID: 426.7443 Class 7 FID: 442.5496 Class 8 FID: 413.6382 Class 9 FID: 375.5112 Avg FID: 346.0624
cosine	2000	Class 0 FID: 454.6992 Class 1 FID: 534.8362 Class 2 FID: 418.8211 Class 3 FID: 331.8481 Class 4 FID: 415.0370 Class 5 FID: 387.9117 Class 6 FID: 426.7443 Class 7 FID: 442.5496 Class 8 FID: 413.6382 Class 9 FID: 375.5112 Avg FID: 420.1597

Table 6: Conditional DDPM with cosine scheduler

References

- [1] Prafulla Dhariwal and Alex Nichol. Diffusion Models Beat GANs on Image Synthesis. 2021. arXiv:2105.05233. [<https://arxiv.org/abs/2105.05233>]
- [2] Allan Jabri, David Fleet, and Ting Chen. Scalable Adaptive Computation for Iterative Generation. 2023. arXiv:2212.11972. [<https://arxiv.org/abs/2212.11972>]
- [3] Alexander C. Li et al. Your diffusion model is secretly a zero-shot classifier. In Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 2206–2217. [<https://arxiv.org/abs/2303.16203>]
- [4] Xiner Li et al. Derivative-free guidance in continuous and discrete diffusion models with soft value-based decoding. arXiv preprint arXiv:2408.08252, 2024. [<https://arxiv.org/abs/2408.08252>]
- [5] Alex Nichol and Prafulla Dhariwal. Improved Denoising Diffusion Probabilistic Models. 2021. arXiv:2102.09672. [<https://arxiv.org/abs/2102.09672>]
- [6] Jacob Austin et al. Structured denoising diffusion models in discrete state-spaces. Advances in Neural Information Processing Systems 34, 2021, pp. 17981–17993. [<https://arxiv.org/abs/2107.03006>]
- [7] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. Advances in Neural Information Processing Systems 33, 2020, pp. 6840–6851. [<https://dl.acm.org/doi/abs/10.5555/3495724.3496298>]
- [8] Leo Zhang. The Cosine Schedule is Fisher-Rao-Optimal for Masked Discrete Diffusion Models. 2025. arXiv:2508.04884. [<https://arxiv.org/abs/2508.04884>]